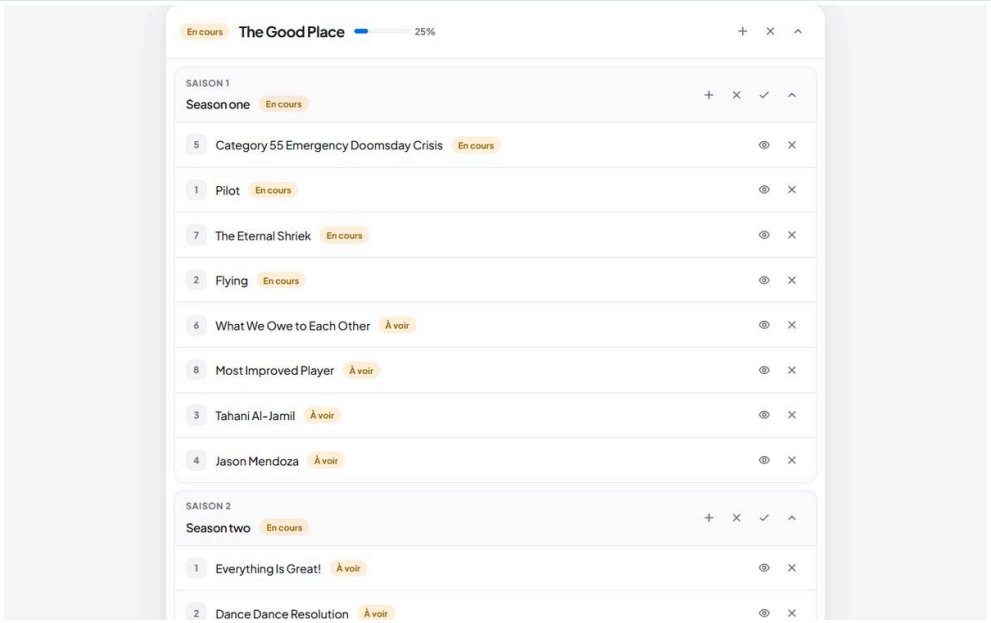
GestionSeries · compte-rendu

TP JEE (DAO/JPA) + API REST Jersey/JWT. Tomcat 10, Jakarta, EclipseLink, MySQL.

UI : /GestionSeries/ · API : /GestionSeries/api/*

Capture UI



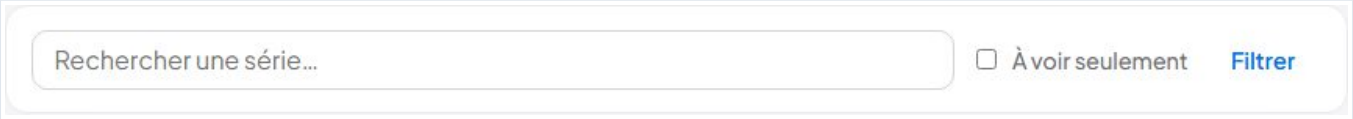
Liste des séries (localhost).

1. TP JEE + bonus UI

- DAO + DaoFactory (Singleton/Factory), JPA, named queries, transactions
- Adaptations : jakarta.*, MySQL, EM par requête, LAZY + JOIN FETCH, flash session
- Bonus : recherche, filtre à voir, progression %, marquer saison vue

Architecture

UI (Servlets/JSP) et API (Jersey) partagent DaoFactory · DAO · EntityManager · MySQL.



Barre recherche / filtre.

2. API REST (Rest API.pdf)

Phase A · Jersey

- Deps jersey-container-servlet-core, jersey-hk2, jackson, json-binding 3.0.1
- web.xml : ServletContainer, package fr.pau.univ.series.services, /api/*

Phase B · endpoints

- GET /api/serie/all et GET /api/serie/{id} (public)
- POST /api/serie/add, /saison/add, /episode/add (admin + token)
- DELETE /api/serie/delete/{id}?token=· (admin)

Phase C · JWT

- User + PasswordHelper + AuthenticationService (POST /api/auth/login)
- Comptes seed : admin/admin (ADMIN), user/user (USER)
- jjwt 0.11.5 (Jakarta) à la place de 0.9.1 du PDF

Exemple · liste JSON

SeriesService.java

```java
@Path("/serie")
public class SeriesService {
  @GET @Path("/all") @Produces(MediaType.APPLICATION_JSON)
  public Response getSeries() {
    List<Serie> ret = DaoFactory.getInstance().getSerieDao().readAllSeries();
    return Response.ok(new GenericEntity<List<Serie>>(ret) {}).build();
  }
}
```

Exemple · login

AuthenticationService.java

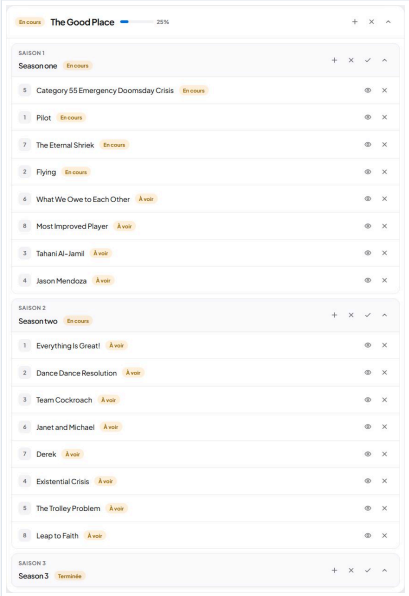
```java
@Path("/auth")
public class AuthenticationService {
  @POST @Path("/login")
  @Consumes(MediaType.APPLICATION_FORM_URLENCODED)
  public Response autenticateUser(MultivaluedMap<String, String> formParams) {
    // vérifie user/pwd · renvoie le JWT
  }
}
```

Postman

- GET http://localhost:8080/GestionSeries/api/serie/all
- POST ·/api/auth/login (user=admin, pwd=admin) · token
- POST ·/api/serie/add (nom=Test, token=<JWT>)

Oral

- DAO + Factory : même couche pour UI et API
- JPA + EM par requête
- REST + JWT : lecture publique, écriture admin



UI détail (coexiste avec l·API).